Git & GitHub

Complete Textbook

From Basics to Advanced

Version Control | Branching | Merging | GitHub | CI/CD

24 Comprehensive Chapters

2025 Edition

Chapter 1: Version Control Basics

1.1 What is Version Control?

Version Control is a system that records changes to a file or set of files over time so that you can recall specific versions later. It allows multiple developers to collaborate, track history, and revert mistakes. Think of it as a complete history of every change ever made to your project.

A Version Control System (VCS) keeps a database of all the changes made to files in a project. Every modification is recorded with metadata: who made it, when, and why. This enables teams to work simultaneously without overwriting each other's work.

1.2 Why Version Control?

Version control is essential in modern software development for several critical reasons:

- Collaboration: Multiple developers can work on the same project simultaneously without conflicts.
- History: Every change is tracked — you can see who changed what and when.
- Revert: Mistakes can be undone by reverting to any previous version.
- Branching: Developers can experiment in isolated branches without affecting the main codebase.
- Backup: The entire project history is stored, providing a natural backup mechanism.
- Accountability: Clear audit trail of all modifications made to the codebase.
- Release Management: Specific versions can be tagged and released to users.

 Key Insight: Without version control, teams rely on manually copying files and emailing code — a fragile, error-prone process that does not scale. Version control is the foundation of professional software development.

1.3 Types of Version Control Systems

1.3.1 Local Version Control System (LVCS)

The simplest form of version control. Changes are stored in a local database on the developer's machine. The most well-known example is RCS (Revision Control System).

- Stores file differences (deltas) in a special database format.
- Works only on the local machine — no network required.
- Major drawback: If the hard disk is corrupted, all history is lost.
- No support for collaboration — only one developer can work on a project.

1.3.2 Centralized Version Control System (CVCS)

Centralized VCS (e.g., SVN, CVS, Perforce) stores all versioned files on a central server. Developers check out files from this central server.

- Single source of truth: all history on one server.
- Administrators can control access and permissions centrally.

- Drawback: The central server is a single point of failure.
- Requires network connectivity to commit changes.
- If the server goes down, no one can commit or retrieve history.

1.3.3 Distributed Version Control System (DVCS)

In a DVCS (e.g., Git, Mercurial), every developer has a full copy of the entire repository, including all history. There is no single point of failure.

- Every clone is a full backup of the entire repository.
- Work offline — commit, branch, and view history without a network.
- Faster operations — most commands run locally.
- Multiple workflows supported: centralized, feature branch, fork-based.

Feature	Centralized VCS	Distributed VCS
Storage	Central server only	Every developer's machine
Offline Work	Limited	Fully supported
Speed	Network dependent	Mostly local (fast)
Backup	Single point of failure	Every clone is a backup
Branching	Expensive/slow	Lightweight and fast
Examples	SVN, CVS, Perforce	Git, Mercurial

1.4 Git vs Other VCS

Git has become the dominant version control system in the world. Here is how it compares to other popular systems:

Aspect	Git vs SVN / CVS
Architecture	Distributed (full local repo) vs Centralized (server-only)
Branching	Lightweight, instant vs Expensive directory copies
Speed	Fast (local operations) vs Slower (network calls)
Data Model	Snapshots of files vs Differences (deltas)
Offline	Full functionality offline vs Requires server connection
Integrity	SHA-1 checksums for all data vs No built-in data verification

Chapter 2: Git Basics

2.1 What is Git?

Git is a free and open-source distributed version control system designed to handle everything from small to very large projects with speed and efficiency. Created by Linus Torvalds in 2005 for Linux kernel development, Git has become the industry standard for version control.

Git thinks about data differently from other VCS tools. Instead of storing a list of file-based changes (deltas), Git thinks of its data as a series of snapshots of a miniature filesystem. Every time you commit, Git takes a picture of what all your files look like and stores a reference to that snapshot.

2.2 Git Features

- Speed: Most operations are performed locally, making Git extremely fast.
- Data Integrity: Everything in Git is checksummed using SHA-1 before it is stored.
- Non-linear Development: Git supports thousands of parallel branches.
- Fully Distributed: Every clone is a full repository with complete history.
- Open Source: Free to use, with a huge community and ecosystem.
- Staging Area: Unique intermediate area allowing fine-grained commits.
- Branching Model: Lightweight branches enable safe experimentation.

2.3 Distributed Nature of Git

Unlike centralized systems, in Git every developer has a complete local copy of the entire project history. This means:

- You can commit, branch, and merge entirely offline.
- Operations like log, diff, and blame do not require network access.
- Collaboration happens by pushing to and pulling from shared repositories.
- Multiple remote repositories can be used simultaneously (origin, upstream, etc.).

2.4 Git Workflow

The standard Git workflow follows these steps:

1. Modify files in your Working Directory.
2. Stage the files you want to include in your next commit (git add).
3. Commit the staged snapshot permanently to your local repository (git commit).
4. Push commits to a remote repository to share with the team (git push).
5. Pull changes from the remote to keep your local copy up to date (git pull).

 Workflow Summary: Working Directory → Staging Area (Index) → Local Repository → Remote Repository. Each stage represents a level of permanence and shareability.

2.5 Git Terminologies

Term	Definition	Example
Repository (Repo)	A directory tracked by Git containing all project files and history	my-project/.git/
Commit	A saved snapshot of staged changes with a unique SHA-1 hash	git commit -m 'Add login'
Branch	A lightweight pointer to a specific commit	main, feature/login
HEAD	A pointer to the current branch's latest commit	refs/heads/main
Remote	A version of the repository hosted on the internet or network	origin, upstream
Clone	A full copy of a remote repository on your local machine	git clone <url>
Fork	A personal copy of another user's repository on GitHub	Fork button on GitHub
Merge	Integrating changes from one branch into another	git merge feature-branch
Rebase	Moving or combining commits to a new base commit	git rebase main
Tag	A named reference to a specific commit (usually a release)	v1.0.0, v2.3.1
Index	Another name for the Staging Area	git add adds to Index
Blob	Git's internal object representing file content	Stored in .git/objects

Chapter 3: Git Architecture (Internal Working)

Understanding Git's internal architecture helps you use it more effectively and debug issues confidently. Git operates on three main areas on your local machine plus the commit history.

3.1 Working Directory

The Working Directory (also called the Working Tree) is the directory on your filesystem where you edit files. It contains the actual project files that you see and modify. Git tracks the difference between your working directory and the staging area.

- Contains checked-out files from the latest commit.
- Files here can be in two states: tracked (known to Git) or untracked (new files Git doesn't know about).
- Changes here are not yet saved to Git until staged and committed.

3.2 Staging Area (Index)

The Staging Area (also called the Index) is a preparation zone between your working directory and the repository. It allows you to carefully craft each commit by selecting exactly which changes to include.

- Located in the .git/index file.
- Acts as a draft for your next commit.
- You can stage partial file changes using git add -p.
- Allows you to group related changes into meaningful commits.

 Why Staging? The staging area is Git's unique feature that allows you to carefully prepare commits. You can make multiple changes but only commit a subset, keeping your commit history clean and meaningful.

3.3 Local Repository

The Local Repository is the .git directory in your project root. It stores all the project metadata, history, objects (blobs, trees, commits), and references (branches, tags). When you run git commit, the staged snapshot is permanently saved here.

- Contains: objects (all data), refs (branches/tags), config, HEAD, logs.
- git init creates a new .git directory.
- All history is compressed and stored efficiently using SHA-1 hashes.

3.4 Commit History

Git's commit history is a directed acyclic graph (DAG) of commit objects. Each commit points to its parent commit(s), forming a chain of history. Merge commits have two parents. This structure allows Git to efficiently compute differences and traverse history.

- Each commit stores: tree (snapshot), parent(s), author, committer, message.
- SHA-1 hash uniquely identifies every commit.
- git log traverses this graph from HEAD backwards.

3.5 HEAD

HEAD is a special pointer in Git that indicates the currently checked-out commit or branch. It is stored in the .git/HEAD file. HEAD usually points to a branch name, which in turn points to the latest commit on that branch.

- Attached HEAD: points to a branch (normal state) — e.g., ref: refs/heads/main.
- Detached HEAD: points directly to a commit hash (occurs after git checkout <hash>).
- When you commit, HEAD (and the branch it points to) moves forward automatically.

```
# View current HEAD
cat .git/HEAD
# Output: ref: refs/heads/main

# Detached HEAD state (checking out a specific commit)
git checkout abc1234
# HEAD is now at abc1234
```

Chapter 4: Git Installation & Configuration

4.1 Git Installation

Git can be installed on all major operating systems. Choose the method appropriate for your platform:

Linux (Debian/Ubuntu)

```
sudo apt-get update
sudo apt-get install git
```

Linux (Fedora/RHEL/CentOS)

```
sudo dnf install git
```

macOS

```
# Using Homebrew
brew install git

# Or install Xcode Command Line Tools (includes git)
xcode-select --install
```

Windows

Download the installer from https://git-scm.com/download/win and run it. This installs Git Bash (a Unix-like terminal) along with Git.

```
# Verify installation
git --version
# Output: git version 2.43.0
```

4.2 git config

After installing Git, the first step is to configure your identity. Git uses this information for every commit you make. Configuration is stored at three levels:

Level	Description & File Location
System	Applies to all users on the machine — /etc/gitconfig
Global	Applies to all repos for current user — ~/.gitconfig
Local	Applies only to the current repository — .git/config

4.3 Username & Email Configuration

You must set your name and email before making any commits. Git embeds this information into every commit:

```
# Set global username
git config --global user.name "Your Name"

# Set global email
git config --global user.email "you@example.com"

# View all configuration settings
git config --list

# View a specific setting
git config user.name
```

4.4 Global vs Local Config

Global configuration applies to all repositories on your machine. Local configuration overrides global settings for a specific repository, which is useful when working with different identities (e.g., personal vs work):

```
# Set local config (inside a specific repo)
git config --local user.email "work@company.com"

# Other useful global settings
git config --global core.editor "code --wait"   # VS Code as editor
git config --global init.defaultBranch main      # Default branch name
git config --global color.ui auto                # Colored output
git config --global alias.st status              # Create alias 'git st'
```

Chapter 5: Git Repository

5.1 git init

The git init command initializes a new Git repository in the current directory. It creates a .git subdirectory containing all the necessary repository files and metadata.

```
# Initialize a new repository in the current directory
git init

# Initialize a new repository in a specified directory
git init my-project
cd my-project

# Initialize a bare repository (for servers)
git init --bare my-repo.git
```

After git init, the .git directory contains:

- HEAD — pointer to the current branch.
- config — repository-specific configuration.
- objects/ — all data (blobs, trees, commits, tags).
- refs/ — pointers to commit objects (branches, tags, remotes).

5.2 git clone

The git clone command creates a copy of an existing repository. It copies all files, history, and branches from the source repository to your local machine.

```
# Clone a remote repository
git clone https://github.com/username/repo.git

# Clone into a specific directory
git clone https://github.com/username/repo.git my-folder

# Clone a specific branch only
git clone -b develop https://github.com/username/repo.git

# Shallow clone (only recent history, faster)
git clone --depth 1 https://github.com/username/repo.git
```

Note: git clone automatically sets up the remote named 'origin' pointing to the cloned URL. It also sets up tracking branches so git pull and git push work without additional configuration.

5.3 Bare vs Non-bare Repository

Git repositories come in two types, each serving different purposes:

Non-bare Repository	Bare Repository
Has a working directory with files you can edit	No working directory — only the .git contents
Used for active development	Used as a shared remote/central repository
Contains .git/ folder inside project folder	The repository IS the .git folder (typically named repo.git)
Created with: git init	Created with: git init --bare
You can make commits directly	Not meant for direct commits — only push/pull

Chapter 6: Git File Lifecycle

Every file in your working directory can be in one of four states in Git's lifecycle. Understanding this lifecycle is fundamental to working with Git effectively.

6.1 Untracked

Untracked files are new files that Git does not know about yet. They exist in your working directory but have never been added to the Git index (staging area). Git will not include untracked files in commits.

```
# Create a new file
echo 'Hello World' > newfile.txt

# Git shows it as untracked
git status
# Output: Untracked files: newfile.txt
```

6.2 Modified

A file becomes Modified when it has been changed since the last commit but has not yet been staged. Git tracks these changes but will not include them in the next commit unless you stage them.

```
# Edit an already-tracked file
echo 'New content' >> existing.txt

git status
# Output: Changes not staged for commit: modified: existing.txt
```

6.3 Staged

Staged files are files that have been marked to be included in the next commit. You move files to the staging area using git add. The staging area is a snapshot of what your next commit will look like.

```
# Stage a specific file
git add newfile.txt

# Stage all changes in current directory
git add .

# Stage parts of a file interactively
git add -p filename.txt
```

6.4 Committed

Once files are committed, their current state is permanently saved to the local Git repository as a snapshot. Committed data is considered safe — it is stored in the database and can always be retrieved.

```
# Commit staged files with a message
git commit -m "Add new feature"

# Stage all tracked modified files and commit in one step
```

```
git commit -am "Fix bug in login"
```

State	Description & Transition
Untracked	New file, unknown to Git → git add → Staged
Modified	Tracked file with unsaved changes → git add → Staged
Staged	Marked for next commit → git commit → Committed
Committed	Safely saved in repo → edit file → Modified

Chapter 7: Git Commands (Core)

7.1 git status

git status shows the state of the working directory and staging area. It tells you which changes have been staged, which haven't, and which files are not being tracked.

```
git status

# Short/compact output
git status -s
# M  = modified, A = added/staged, ?? = untracked
```

7.2 git add

git add moves changes from the working directory to the staging area. It does not affect the repository — the change is staged, not committed.

```
git add filename.txt        # Stage a specific file
git add .                   # Stage all changes in current dir
git add *.js               # Stage all .js files
git add -p                 # Interactively stage hunks (partial files)
git add -A                 # Stage all changes including deletions
```

7.3 git commit

git commit saves the staged snapshot permanently to the repository. Every commit requires a message describing what changed and why.

```
git commit -m "Your message here"
git commit -am "Message"     # Stage all tracked + commit
git commit --amend           # Modify the most recent commit
git commit --amend -m "New message"  # Fix last commit message
```

> ✓ **Good Commit Messages:** Use imperative mood: 'Add feature' not 'Added feature'. Keep the subject line under 72 characters. Explain WHY, not just what — the code shows what changed.

7.4 git log

git log shows the commit history. It has many options to control output format and filter commits:

```
git log                     # Full log with all details
git log --oneline           # One line per commit
git log --oneline --graph   # ASCII graph showing branches
git log --all --oneline     # All branches
git log -5                  # Last 5 commits only
git log --author="John"     # Commits by specific author
git log --since="2024-01-01" # Commits after a date
```

```
git log --grep="login"       # Search commit messages
```

7.5 git diff

git diff shows differences between various states of your files:

```
git diff                      # Working dir vs Staging area
git diff --staged             # Staging area vs Last commit
git diff HEAD                # Working dir vs Last commit
git diff branch1 branch2     # Compare two branches
git diff abc123 def456       # Compare two commits
```

7.6 git show

git show displays information about a specific Git object — usually a commit. It shows the commit metadata (author, date, message) and the diff introduced by that commit.

```
git show                      # Show latest commit
git show abc1234             # Show a specific commit
git show HEAD~2              # Show 2 commits before HEAD
git show v1.0.0             # Show a tagged commit
```

Chapter 8: Branching in Git

8.1 What is a Branch?

A branch in Git is simply a lightweight movable pointer to one of the commits. The default branch name in Git is main (or master in older repositories). As you make commits, the branch pointer moves forward automatically to point to the latest commit.

Branching is one of Git's most powerful features. It allows developers to diverge from the main line of development and work on features, bug fixes, or experiments in isolation — without affecting the main codebase.

8.2 Default Branch (main / master)

When you initialize a repository or make the first commit, Git creates a default branch. Historically this was called master, but the industry standard has shifted to main. The default branch is the primary branch of the project.

```
# Set default branch name globally
git config --global init.defaultBranch main
```

8.3 Create Branch

```
# Create a new branch (does not switch to it)
git branch feature-login

# Create and switch to a new branch in one step
git checkout -b feature-login

# Modern way (Git 2.23+)
git switch -c feature-login

# Create branch from a specific commit or tag
git checkout -b hotfix v2.0.0
```

8.4 Switch Branch

```
# Switch to an existing branch
git checkout main

# Modern syntax (Git 2.23+)
git switch main

# List all branches (* = current)
git branch

# List all branches including remotes
git branch -a
```

8.5 Delete Branch

```
# Delete a merged branch (safe)
git branch -d feature-login

# Force delete an unmerged branch
git branch -D feature-login

# Delete a remote branch
git push origin --delete feature-login
```

8.6 Branching Strategies

Teams adopt branching strategies to organize collaboration. Common strategies include:

Git Flow

A robust model with dedicated branches for features, releases, and hotfixes. Uses: main (production), develop (integration), feature/*, release/*, hotfix/*. Best for projects with scheduled releases.

GitHub Flow

Simpler model: main is always deployable. All work happens in short-lived feature branches merged via Pull Requests. Best for continuous deployment.

Trunk-Based Development

Developers commit small, frequent changes directly to the main branch (or very short-lived branches). Uses feature flags to hide incomplete features. Best for large teams practicing CI/CD.

Chapter 9: Merging & Rebasing

9.1 git merge

Merging is Git's way of putting a forked history back together. The git merge command combines multiple sequences of commits into one unified history.

```
# First, switch to the target branch
git checkout main

# Merge the feature branch into main
git merge feature-login

# Merge with a commit (no fast-forward)
git merge --no-ff feature-login

# Abort a merge in progress
git merge --abort
```

9.2 Fast-forward Merge

A fast-forward merge occurs when the target branch has not diverged from the source branch. Git simply moves the branch pointer forward to the latest commit of the merged branch. No merge commit is created — the history remains linear.

> **Fast-Forward Condition:** Fast-forward is possible only when the current branch has no new commits since the feature branch was created. Git just advances the pointer without creating a merge commit.

9.3 Three-way Merge

When both branches have diverged (each has new commits since they diverged), Git performs a three-way merge using three commits: the common ancestor (merge base), the tip of the current branch, and the tip of the branch being merged. Git creates a new merge commit with two parents.

```
# View the merge base (common ancestor)
git merge-base main feature-login
```

9.4 git rebase

Rebasing is an alternative to merging. Instead of creating a merge commit, rebase moves or replays commits from one branch onto another, resulting in a linear history. It rewrites the commit history.

```
# Rebase feature branch onto main
git checkout feature-login
git rebase main

# Interactive rebase (edit, squash, reorder last 3 commits)
git rebase -i HEAD~3
```

```
# Abort a rebase in progress
git rebase --abort

# Continue after resolving conflicts
git rebase --continue
```

9.5 Merge vs Rebase

git merge	git rebase
Preserves complete history	Creates linear, cleaner history
Creates a merge commit	No merge commit — replays commits
Non-destructive (safe for shared branches)	Rewrites history (never use on public branches)
Shows exactly when branches were merged	History appears as if work was done sequentially
Easier to understand what happened	Cleaner project history

 Golden Rule of Rebasing: Never rebase commits that have been pushed to a shared public repository. Rebase rewrites history, which causes problems for everyone who has based work on those commits.

Chapter 10: Git Conflicts

10.1 What is a Merge Conflict?

A merge conflict occurs when Git cannot automatically merge changes from two branches because the same part of a file was changed differently in each branch. Git pauses the merge and asks you to manually resolve the conflicting sections.

10.2 Why Conflicts Occur

- Two developers modified the same line(s) in the same file differently.
- One developer deleted a file while another modified it.
- Two branches added the same filename in the same directory.
- Merge, rebase, or cherry-pick operations that affect the same code regions.

10.3 Resolving Conflicts

When a conflict occurs, Git marks the conflicting section in the file with conflict markers:

```
<<<<<<< HEAD
This is the change from the current branch (HEAD)
=======
This is the change from the branch being merged
>>>>>>> feature-login
```

To resolve the conflict:

6. Open the conflicting file in your editor.
7. Decide which change to keep (or combine both changes).
8. Remove all conflict markers (<<<<<<<, =======, >>>>>>>).
9. Stage the resolved file: git add filename.txt
10. Complete the merge: git commit

```
# Check which files have conflicts
git status

# After resolving, stage the fixed files
git add resolved-file.txt

# Complete the merge
git commit

# Abort and go back to pre-merge state
git merge --abort
```

10.4 Best Practices to Avoid Conflicts

- Pull from the remote frequently to keep your branch up to date with main.
- Keep feature branches small and short-lived — merge quickly.
- Communicate with your team about who is editing which files.
- Modularize code — avoid large files that many people edit simultaneously.
- Use code formatters (Prettier, Black) consistently so formatting differences don't cause conflicts.
- Rebase onto main regularly to integrate upstream changes incrementally.

Chapter 11: Git Reset, Revert & Checkout

11.1 git reset

git reset moves the HEAD pointer and optionally modifies the staging area and working directory. It is used to undo commits or unstage files. There are three modes:

--soft

Moves HEAD to the specified commit. Staged changes and working directory are preserved. The commits are undone but changes remain staged — ready to be re-committed.

```
git reset --soft HEAD~1     # Undo last commit, keep changes staged
```

--mixed (Default)

Moves HEAD and resets the staging area. Working directory is preserved. Changes from the undone commits go back to 'modified' (unstaged) state.

```
git reset HEAD~1            # Undo last commit, unstage changes
git reset HEAD filename     # Unstage a specific file
```

--hard

Moves HEAD, resets the staging area, AND resets the working directory. All changes in the undone commits are permanently discarded. This is destructive and irreversible (unless you use git reflog).

```
git reset --hard HEAD~1     # Undo last commit, discard all changes
git reset --hard origin/main  # Reset to match remote
```

Mode	HEAD | Staging Area | Working Dir
--soft	Moved | Unchanged | Unchanged
--mixed (default)	Moved | Reset | Unchanged
--hard	Moved | Reset | Reset (changes lost!)

11.2 git revert

git revert creates a new commit that undoes the changes from a previous commit. It does NOT rewrite history, making it safe to use on shared/public branches. The original commit remains in history.

```
git revert HEAD             # Revert the last commit
git revert abc1234         # Revert a specific commit
git revert HEAD~3..HEAD    # Revert last 3 commits
```

11.3 git checkout

git checkout is a versatile command used to switch branches or restore working tree files. (In Git 2.23+, git switch and git restore were introduced to separate these concerns.)

```
git checkout main           # Switch to main branch
```

```
git checkout -b newbranch # Create and switch to new branch
git checkout -- filename  # Discard changes in working dir (restore file)
git checkout abc1234      # Detach HEAD to a specific commit
```

11.4 When to Use Which

Command	When to Use
git reset --soft	Undo commit(s) but keep all changes staged — re-commit differently
git reset --mixed	Undo commit(s) and unstage changes — review before re-staging
git reset --hard	Completely discard commits and all changes (destructive)
git revert	Undo a commit safely on a shared/public branch
git checkout -- file	Discard unsaved changes to a specific file in working dir

Chapter 12: Git Stash

12.1 What is Stash?

git stash temporarily shelves (stashes) changes you have made to your working directory so you can work on something else, and then come back and re-apply them later. It is useful when you need to quickly switch context — for example, to fix an urgent bug — without committing incomplete work.

12.2 git stash save

```
# Stash current changes (tracked files)
git stash

# Stash with a descriptive message
git stash save "WIP: half-done login feature"

# Stash including untracked files
git stash -u

# Stash including untracked AND ignored files
git stash -a
```

12.3 git stash list

Each stash is stored as an entry in a stack. git stash list shows all stored stashes:

```
git stash list
# Output:
# stash@{0}: On main: WIP: half-done login feature
# stash@{1}: On feature-x: temp save
```

12.4 git stash apply

git stash apply restores the stashed changes without removing the stash from the list. The changes are reapplied to your working directory.

```
# Apply the most recent stash
git stash apply

# Apply a specific stash
git stash apply stash@{1}
```

12.5 git stash pop

git stash pop applies the most recent stash AND removes it from the stash list. It is the equivalent of git stash apply followed by git stash drop.

```
# Pop (apply and remove) the latest stash
```

```
git stash pop

# Pop a specific stash
git stash pop stash@{2}

# Remove a stash without applying it
git stash drop stash@{0}

# Clear all stashes
git stash clear
```

 Tip: Use git stash branch <branchname> to create a new branch from a stash. This applies the stash to the new branch and drops it from the stash list — useful when a stash conflicts with your current branch.

Chapter 13: Git Tags

Tags are references that point to specific commits in Git history. They are typically used to mark release points (e.g., v1.0.0, v2.3.1). Unlike branches, tags do not move — they always point to the same commit.

13.1 Lightweight Tags

A lightweight tag is simply a pointer to a specific commit — like a branch that does not change. It stores no extra information. Use for temporary or personal markers.

```
# Create a lightweight tag
git tag v1.0

# Tag a specific commit
git tag v1.0 abc1234

# List all tags
git tag

# List tags matching a pattern
git tag -l "v2.*"
```

13.2 Annotated Tags

Annotated tags are stored as full objects in the Git database. They contain the tagger's name, email, date, a message, and can be signed with GPG. Recommended for official releases.

```
# Create an annotated tag
git tag -a v2.0.0 -m "Release version 2.0.0"

# View tag details
git show v2.0.0

# Tag an older commit with annotation
git tag -a v1.5 abc1234 -m "Version 1.5 (retroactive)"

# Push a specific tag to remote
git push origin v2.0.0

# Push all tags to remote
git push origin --tags

# Delete a local tag
git tag -d v1.0

# Delete a remote tag
git push origin --delete v1.0
```

13.3 Versioning Releases

The most common versioning convention is Semantic Versioning (SemVer): MAJOR.MINOR.PATCH

- MAJOR: Incremented for incompatible API changes (breaking changes).
- MINOR: Incremented for new backward-compatible functionality.
- PATCH: Incremented for backward-compatible bug fixes.
- Pre-release: e.g., v2.0.0-alpha, v2.0.0-beta.1, v2.0.0-rc.1

Chapter 14: Git Ignore & Clean

14.1 .gitignore

The .gitignore file tells Git which files and directories to ignore — to not track and not include in commits. This is essential for excluding build artifacts, dependency directories, secrets, and OS-specific files.

```
# .gitignore example for a Node.js project

# Dependency directories
node_modules/

# Build output
dist/
build/

# Environment variables (never commit secrets!)
.env
.env.local

# OS-generated files
.DS_Store
Thumbs.db

# IDE files
.vscode/
.idea/

# Log files
*.log
logs/
```

14.2 Ignoring Files — Pattern Rules

- Lines starting with # are comments.
- A trailing / matches a directory only: dist/ ignores the dist directory.
- A leading / anchors to the repository root: /config.js ignores only root config.js.
- An asterisk (*) matches anything except a slash: *.log ignores all .log files.
- Double asterisk (**) matches across directories: **/logs ignores logs anywhere.
- A leading ! negates the pattern: !important.log un-ignores this specific file.

 Important: .gitignore only affects untracked files. If a file was already committed, adding it to .gitignore will NOT stop tracking it. Use git rm --cached filename to stop tracking an already-committed file.

```
# Stop tracking a committed file
git rm --cached .env
echo '.env' >> .gitignore
git commit -m "Stop tracking .env"
```

14.3 git clean

git clean removes untracked files from your working directory. It is useful for cleaning up build artifacts or generated files that are not tracked by Git.

```
# Show what would be deleted (dry run)
git clean -n

# Remove untracked files
git clean -f

# Remove untracked files AND directories
git clean -fd

# Remove untracked AND ignored files
git clean -fdx
```

 Warning: git clean is destructive — it permanently deletes files that are not tracked. Always run git clean -n (dry run) first to preview what will be deleted before actually running it.

Chapter 15: Git Logs & History

15.1 git log Options

git log is one of the most powerful Git commands for exploring history. It supports a rich set of options for filtering and formatting output:

```
# Full log
git log

# One-line summary per commit
git log --oneline

# Graph showing branches and merges
git log --oneline --graph --all

# Compact graph with decoration
git log --oneline --decorate --graph --all

# Custom format
git log --pretty=format:"%h %an %ar - %s"
# %h=short hash, %an=author, %ar=relative time, %s=subject

# Filter by author
git log --author="Alice"

# Filter by date range
git log --since="2024-01-01" --until="2024-12-31"

# Search in commit messages
git log --grep="bugfix"

# Show which files changed
git log --stat

# Show actual diffs
git log -p

# Commits that changed a specific file
git log -- filename.txt
```

15.2 git reflog

git reflog (reference log) records every position that HEAD has pointed to. It is Git's safety net — it lets you recover commits even after a reset --hard or an accidental branch deletion.

```
# Show reflog
git reflog

# Typical output:
```

```
# abc1234 HEAD@{0}: commit: Add feature
# def5678 HEAD@{1}: reset: moving to HEAD~1
# ghi9012 HEAD@{2}: checkout: moving from main to feature

# Recover a lost commit
git checkout HEAD@{2}
# or
git reset --hard HEAD@{2}
```

15.3 Commit History Tracking

Beyond git log, Git provides tools to trace the history of specific lines and files:

```
# Show who last modified each line of a file
git blame filename.txt

# Blame a specific line range
git blame -L 10,20 filename.txt

# Find when a bug was introduced (binary search)
git bisect start
git bisect bad                 # Current commit is bad
git bisect good v1.0.0         # This version was good
# Git checks out the middle commit
# Test and mark as good or bad
git bisect good / git bisect bad
git bisect reset               # Return to original state
```

Chapter 16: Git Remote Repositories

16.1 What is a Remote?

A remote repository is a version of your project hosted on the internet or a network. Remotes allow team members to share code. The most common remote is named origin, which is automatically set up when you clone a repository.

16.2 git remote

```
# List configured remotes
git remote

# List remotes with URLs
git remote -v

# Add a new remote
git remote add origin https://github.com/user/repo.git

# Add an upstream remote (for forks)
git remote add upstream https://github.com/original/repo.git

# Change remote URL
git remote set-url origin https://new-url.git

# Remove a remote
git remote remove origin
```

16.3 git fetch

git fetch downloads all new data (commits, branches, tags) from the remote but does NOT integrate (merge) them into your local branches. It updates the remote-tracking branches (e.g., origin/main) but leaves your working directory untouched.

```
# Fetch from origin
git fetch origin

# Fetch from all remotes
git fetch --all

# Fetch and prune deleted remote branches
git fetch -p
```

16.4 git pull

git pull is shorthand for git fetch followed by git merge. It downloads new data from the remote AND immediately merges it into your current local branch.

```
# Pull from tracked remote branch
git pull

# Pull from specific remote and branch
git pull origin main

# Pull with rebase instead of merge
git pull --rebase
```

16.5 git push

git push uploads your local commits to the remote repository, making them available to other team members.

```
# Push current branch to its tracked remote
git push

# Push and set upstream tracking
git push -u origin main

# Push a specific branch
git push origin feature-login

# Force push (DANGEROUS — rewrites remote history)
git push --force

# Safer force push (fails if remote has changed)
git push --force-with-lease
```

 Warning: Never force-push to shared branches (main, develop). Force push rewrites history on the remote, causing serious problems for all collaborators who have based their work on the old history.

Chapter 17: GitHub Basics

17.1 What is GitHub?

GitHub is a cloud-based hosting service for Git repositories. It adds a web-based interface and collaboration features on top of Git: pull requests, issues, code review, project management, Actions (CI/CD), and more. Founded in 2008 and acquired by Microsoft in 2018, GitHub hosts over 100 million repositories.

17.2 GitHub vs Git

Git	GitHub
Version control system (the tool)	Hosting platform for Git repositories
Command-line tool installed locally	Web-based interface accessible via browser
Free, open source software	Freemium service (free public repos, paid features)
Works offline	Requires internet connection
No collaboration features built-in	Pull requests, issues, code review, teams
By Linus Torvalds (2005)	By Tom Preston-Werner et al. (2008)

17.3 GitHub Repositories

A GitHub repository contains all your project files and each file's revision history. Repositories can contain folders and files, images, videos, spreadsheets, and data sets.

- README.md: A markdown file displayed on the repository homepage — explains the project.
- LICENSE: Specifies how others may use your code.
- CONTRIBUTING.md: Guidelines for contributors.
- Code, Issues, Pull Requests, Actions, Projects, Wiki, Insights — all accessible from the repo page.

17.4 Public vs Private Repos

Public Repository	Private Repository
Visible to everyone on the internet	Only visible to you and people you invite
Anyone can fork and contribute via PR	Only collaborators can access
Open source projects	Proprietary or sensitive projects
Free for all plans	Free for individuals; teams may have limits
Indexed by search engines	Not indexed — not discoverable

Chapter 18: GitHub Workflow

The standard GitHub workflow follows a structured process for contributing to projects. This applies both to open source contributions (fork-based) and internal team contributions.

18.1 Fork

Forking creates a personal copy of another user's repository in your own GitHub account. You have full control over your fork and can make any changes. Forks are used to contribute to projects you do not have write access to.

```
# Fork: Use the 'Fork' button on GitHub website
# This creates github.com/YOUR-USERNAME/repo-name
```

18.2 Clone

Clone your fork to your local machine to start working on it:

```
git clone https://github.com/YOUR-USERNAME/repo-name.git
cd repo-name

# Add the original repo as upstream
git remote add upstream https://github.com/ORIGINAL/repo-name.git
```

18.3 Branch

Always create a new branch for your changes. Never commit directly to main:

```
git checkout -b feature/my-new-feature
```

18.4 Commit

Make your changes and commit them with clear, descriptive messages:

```
git add .
git commit -m "Add: implement user authentication"
```

18.5 Push

Push your branch to your fork on GitHub:

```
git push origin feature/my-new-feature
```

18.6 Pull Request (PR)

Go to the GitHub website and open a Pull Request from your branch to the original repository's main branch. Describe your changes, reference related issues, and request reviewers.

 PR Best Practices: Keep PRs small and focused (one feature or bug fix per PR). Write a clear description explaining what changed and why. Reference related issues with 'Closes #123'. Add screenshots for UI changes.

Keeping Your Fork Updated

```
# Fetch latest from upstream
git fetch upstream

# Merge upstream changes into your local main
git checkout main
git merge upstream/main

# Push updated main to your fork
git push origin main
```

Chapter 19: Pull Requests (PR)

19.1 What is a Pull Request?

A Pull Request (PR) is a mechanism for a developer to notify team members that they have completed a feature or bug fix and it is ready for review. It is a request to pull (merge) your branch into another branch. PRs are the central collaboration tool in GitHub — they enable code review, discussion, and quality control before changes reach the main branch.

19.2 Code Review

Code review is the process where other developers examine the changes in a PR to catch bugs, suggest improvements, enforce standards, and share knowledge. GitHub provides inline commenting on specific lines of code.

- Review each file's changes carefully.
- Leave inline comments on specific lines with suggestions or questions.
- Check for correctness, readability, performance, and security issues.
- Suggest improvements with the 'Suggestion' feature (one-click apply).

19.3 Approvals

Before a PR can be merged, it typically requires one or more approvals from designated reviewers. Branch protection rules can enforce a minimum number of required approvals. Reviewers can: Approve (looks good), Request Changes (needs fixing), or Comment (neutral feedback).

19.4 Merge Options

GitHub offers three ways to merge a Pull Request:

Merge Commit

Creates a merge commit that joins the two branch histories. Preserves complete history and shows exactly when branches were merged. The branch history is kept intact.

```
# Equivalent git command
git merge --no-ff feature-branch
```

Squash and Merge

Combines all commits from the PR into a single commit before merging. Results in a clean, linear history on the main branch. Individual commits from the feature branch are lost. Good for keeping history readable.

```
# Equivalent git command
git merge --squash feature-branch
```

Rebase and Merge

Replays each commit from the PR branch onto the base branch individually. Creates a linear history without a merge commit. Commit history is preserved but commit SHAs are rewritten.

```
# Equivalent git command
git rebase main
git merge --ff-only feature-branch
```

Merge Option	Use When
Merge Commit	You want to preserve the full history and see exactly when features landed
Squash and Merge	The feature branch has many messy WIP commits — you want one clean commit
Rebase and Merge	You want linear history AND want to preserve individual commits

Chapter 20: GitHub Branch Protection

Branch protection rules enforce quality controls on important branches (main, develop, release). They prevent accidental or unauthorized changes from reaching critical branches.

20.1 Protected Branches

Protected branches cannot be deleted or force-pushed without bypassing protection. To enable: Repository Settings → Branches → Add Branch Protection Rule. Specify the branch name or pattern (e.g., main, release/*).

20.2 Required Reviews

You can require a minimum number of Pull Request reviews before merging. Options include:

- Require at least N approving reviews before merging (e.g., 1 or 2).
- Dismiss stale pull request approvals when new commits are pushed.
- Require review from Code Owners (defined in CODEOWNERS file).
- Restrict who can dismiss pull request reviews.

20.3 Status Checks

Require that specific CI checks pass before a PR can be merged. This ensures tests, linting, and builds are green before code enters the main branch.

- Require status checks to pass before merging.
- Mark specific checks as required (e.g., CI / build, CI / test).
- Require branches to be up to date before merging.

20.4 Prevent Direct Push

Protecting a branch prevents developers from pushing commits directly to it — all changes must go through a Pull Request. This enforces code review for all changes.

- Require a pull request before merging.
- Include administrators (even admins must use PRs).
- Restrict who can push to matching branches.
- Block force pushes — prevents history rewriting.
- Block deletions — prevents accidental branch deletion.

 CODEOWNERS File: Create a .github/CODEOWNERS file to automatically request reviews from the right people. Example: '*.js @frontend-team' means any JS file change requires approval from the frontend team.

Chapter 21: GitHub Issues & Projects

21.1 Issues

GitHub Issues are the primary way to track bugs, feature requests, tasks, and improvements in a project. Every repository has an Issues tab. Issues support markdown formatting, code blocks, images, file attachments, and cross-references.

- Create an issue with a title, description, and optional metadata.
- Assign to team members responsible for fixing or implementing.
- Link to Pull Requests: PRs that reference an issue automatically close it when merged.
- Use 'Closes #123' in a PR description to auto-close issue #123 on merge.

21.2 Labels

Labels are colored tags that categorize and filter issues and PRs. GitHub creates default labels but you can customize them:

- bug — Something is not working correctly.
- enhancement — A new feature or request.
- good first issue — Good for newcomers to the project.
- help wanted — Extra attention is needed.
- documentation — Improvements or additions to docs.
- question — Further information is requested.
- wontfix — This will not be worked on.

21.3 Milestones

Milestones group issues and PRs together to track progress toward a specific goal, feature, or release. Each milestone shows completion percentage based on open/closed issues.

- Create milestones for upcoming releases (e.g., 'v2.0 Release').
- Assign issues and PRs to milestones.
- Set a due date for the milestone.
- Track overall progress as issues are closed.

21.4 GitHub Projects (Boards)

GitHub Projects provide kanban-style project management boards directly in GitHub. Issues and PRs can be added as cards and moved between columns (e.g., To Do, In Progress, Done).

- Create a Project board linked to your repository.
- Add columns representing workflow stages.
- Add issues and PRs as cards — they stay in sync with their actual status.
- Automate card movement with triggers (e.g., move to Done when PR merges).
- GitHub Projects (new version) supports table view, roadmap view, and custom fields.

Chapter 22: GitHub Actions (CI/CD)

22.1 What is GitHub Actions?

GitHub Actions is a CI/CD (Continuous Integration / Continuous Deployment) platform built directly into GitHub. It automates your software development workflows — building, testing, and deploying your code automatically whenever certain events occur (like a push or pull request).

22.2 Workflow

A workflow is an automated process defined in a YAML file stored in the .github/workflows/ directory. Each YAML file defines one workflow. A workflow is triggered by one or more events and contains one or more jobs.

```
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
```

22.3 Jobs

A job is a set of steps that runs on the same runner. Jobs run in parallel by default but can depend on each other using the 'needs' keyword. Each job runs in a fresh virtual machine environment.

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build project
        run: npm run build

  test:
    needs: build          # Runs after build completes
    runs-on: ubuntu-latest
    steps:
      - run: npm test
```

22.4 Steps

Steps are individual tasks within a job. A step can run a shell command (run:) or use a reusable action (uses:). Steps run sequentially within a job.

```
steps:
  - name: Checkout code
    uses: actions/checkout@v3
```

```
  - name: Setup Node.js
    uses: actions/setup-node@v3
    with:
      node-version: '20'

  - name: Install dependencies
    run: npm install

  - name: Run tests
    run: npm test
```

22.5 Runners

A runner is a server that runs your workflows. GitHub provides hosted runners (Ubuntu, Windows, macOS) or you can host your own self-hosted runners for custom environments or better performance.

```
runs-on: ubuntu-latest    # GitHub-hosted Ubuntu
runs-on: windows-latest   # GitHub-hosted Windows
runs-on: macos-latest     # GitHub-hosted macOS
runs-on: self-hosted      # Your own server
```

22.6 CI/CD Pipelines

A complete CI/CD pipeline using GitHub Actions typically includes these stages:

11. Lint: Check code style and syntax (ESLint, Prettier, Flake8).
12. Test: Run unit, integration, and end-to-end tests.
13. Build: Compile or bundle the application.
14. Security Scan: Check for vulnerabilities (Dependabot, CodeQL).
15. Deploy to Staging: Deploy to a staging environment for QA.
16. Deploy to Production: Deploy to production after approval.

Chapter 23: GitHub Security

23.1 SSH Keys

SSH keys provide a secure way to authenticate with GitHub without entering your username and password. They use public-key cryptography: you keep the private key on your machine and upload the public key to GitHub.

```
# Generate an SSH key pair
ssh-keygen -t ed25519 -C "your_email@example.com"

# Start the SSH agent
eval "$(ssh-agent -s)"

# Add private key to agent
ssh-add ~/.ssh/id_ed25519

# Copy public key to clipboard
cat ~/.ssh/id_ed25519.pub
# Paste into: GitHub → Settings → SSH and GPG Keys → New SSH Key

# Test SSH connection
ssh -T git@github.com
```

23.2 Personal Access Tokens (PAT)

Personal Access Tokens are an alternative to passwords for authenticating to GitHub via HTTPS. As of 2021, GitHub requires tokens instead of passwords for Git operations. Tokens have specific scopes and can be set to expire.

- Generate: GitHub → Settings → Developer Settings → Personal Access Tokens.
- Choose between Classic tokens (broad) or Fine-grained tokens (repository-specific).
- Set scopes: repo (full repo access), workflow, write:packages, etc.
- Store securely — treat a token like a password. Never commit it to code.

23.3 Secrets

GitHub Secrets securely store sensitive information (API keys, passwords, tokens) for use in GitHub Actions workflows. Secrets are encrypted and never exposed in logs.

```
# Access a secret in a workflow
env:
  API_KEY: ${{ secrets.API_KEY }}
  DATABASE_URL: ${{ secrets.DATABASE_URL }}
```

- Store secrets: Repository → Settings → Secrets and variables → Actions.
- Organization secrets can be shared across multiple repositories.
- Environment secrets are scoped to specific deployment environments.

23.4 Dependabot

Dependabot automatically scans your dependencies for known security vulnerabilities and creates pull requests to update vulnerable packages to secure versions. It reads your package files (package.json, requirements.txt, etc.) and checks the GitHub Advisory Database.

- Enable in repository Settings → Security → Dependabot.
- Dependabot alerts: notifies you of vulnerable dependencies.
- Dependabot security updates: auto-creates PRs to fix vulnerabilities.
- Dependabot version updates: keeps dependencies up to date generally.

23.5 Code Scanning

GitHub Code Scanning uses CodeQL (GitHub's static analysis engine) to automatically analyze your code for security vulnerabilities and coding errors. It integrates with GitHub Actions and shows alerts directly in the Security tab and in PRs.

```
# Add to .github/workflows/codeql.yml
- uses: github/codeql-action/init@v3
  with:
    languages: javascript, python
- uses: github/codeql-action/analyze@v3
```

Chapter 24: GitHub Releases

24.1 Versioning

GitHub Releases provide a way to package and distribute software to users. Releases are based on Git tags and include release notes and downloadable assets. They represent meaningful checkpoints in your project's development, usually corresponding to deployable versions.

The recommended versioning standard is Semantic Versioning (SemVer) in the format: MAJOR.MINOR.PATCH

- MAJOR version: Incompatible API changes that break existing functionality.
- MINOR version: New features added in a backward-compatible manner.
- PATCH version: Backward-compatible bug fixes only.
- Pre-releases: v1.0.0-alpha, v1.0.0-beta.1, v1.0.0-rc.1

24.2 Release Notes

Release notes document what changed between versions. Good release notes help users and developers understand the impact of updating. They should be clear, organized, and audience-appropriate.

- What's New: List new features and improvements.
- Bug Fixes: What was broken and is now fixed.
- Breaking Changes: Anything that requires action from users/developers.
- Full Changelog: Link to a comparison showing all commits since last release.

 Tip: Use GitHub's auto-generate release notes feature. It automatically creates notes from merged PRs and new contributors since the last release. Go to: Repository → Releases → Draft a new release → Generate release notes.

24.3 Tags

A release must be based on a Git tag. You can create the tag directly through the GitHub Releases UI or create it locally and push it first:

```
# Create and push an annotated tag
git tag -a v2.0.0 -m "Release v2.0.0"
git push origin v2.0.0

# Then create a release from this tag on GitHub UI
# Repository → Releases → Draft a new release → Choose tag
```

24.4 Assets

Release assets are files attached to a GitHub Release for users to download. For compiled languages or packaged applications, you provide pre-built binaries or packages as assets.

- Automatically included: Source code as .zip and .tar.gz (generated by GitHub).

- Manual uploads: Binaries (.exe, .dmg, .AppImage), compiled packages, Docker images.
- GitHub Actions automation: Use actions/upload-release-asset to upload assets automatically as part of your release pipeline.

```
# Upload asset in GitHub Actions workflow
- name: Upload Release Asset
  uses: actions/upload-release-asset@v1
  with:
    upload_url: ${{ steps.create_release.outputs.upload_url }}
    asset_path: ./dist/myapp-v2.0.0-linux.tar.gz
    asset_name: myapp-v2.0.0-linux.tar.gz
    asset_content_type: application/gzip
```

—— End of Textbook ——